CIS 5500 · FINAL PROJECT

Axiom Shopping Assistant

A relational shopping workflow over **1.4 M Amazon products** and **3.0 M linked reviews**, with database-backed search, evidence, savings, analytics, and weighted ranking.

Team 45 · Chenguang Shen · Luis Garcia · Leo Tan · Anika Madan

DATABASE AND INFORMATION SYSTEMS · UNIVERSITY OF PENNSYLVANIA · SPRING 2026

Problem and goal

Amazon product pages expose **prices, list prices, star ratings, review counts, and seller labels** — but those signals sit side by side, never synthesized. The shopper still has to compare alternatives manually, judge whether a discount is meaningful, and decide which review signals are trustworthy.

THE USER PROBLEM

The shopper still has to:

- compare alternatives manually,
- judge whether a discount is meaningful,
- decide which review signals are trustworthy.

THE PROJECT GOAL

Turn noisy product and review data into a **normalized PostgreSQL database** and a database-backed shopping workflow that answers the questions a shopper actually asks.

WORKFLOW	DATABASE-BACKED ANSWER
Search & browse	keyword, category, brand, price, rating
Product evidence	rating distribution, helpful reviews, alternatives
Cart savings	aggregate list vs. current price savings
Analytics	category, brand, monthly review trends
Value ranking	weighted rating · depth · price · recency

Application map — eight database-backed pages

STACK

Client. React + Vite, custom SVG charts, query-mode toggle.

API. Express + `pg` pool, validation, TTL cache, optimized vs. original SQL selection.

Database. PostgreSQL on AWS RDS — base tables, indexes, materialized views.

Pipeline. Python · pandas · psycopg2 — clean, ER, ingest.

PAGE	PRIMARY ROUTES
Home	`/products/search`, `/deals`, `/products/trending`
Browse	`/categories`, `/products/search`, `/products/category`
Deals	`/deals` (discount sort)
Category / Brand	`/products/category`, `/products/brand`
Product Detail	`/products/:asin` + rating-distribution / helpful-reviews / alternatives
Cart	`POST /cart/savings`
Analytics	`/analytics/categories/compare` · `/brands/performance` · `/reviews/trend`
Value Rankings	`/products/value-rankings` (4 weight params)

Two large overlapping data sources

Amazon Products 2023 — Kaggle, asaniczka

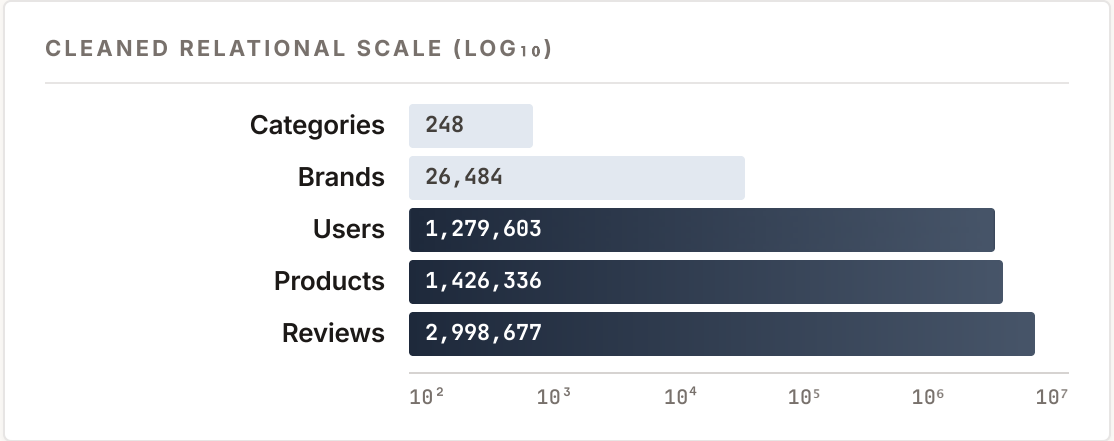
ASIN, title, image / product URLs, price, list price, stars, review count, best-seller flag, category ID.

Amazon Reviews '23 — McAuley Lab, Pure-IDs 5-Core + comment export

Rating, text, helpful votes, verified flag, timestamp, user ID, ASIN, parent ASIN.

Shared keys. `asin` · `parent_asin` · `category_id` · `user_id` — the overlap that makes cross-dataset queries possible.

> *Both fact tables clear the rubric's 100 k threshold by more than 10×.*

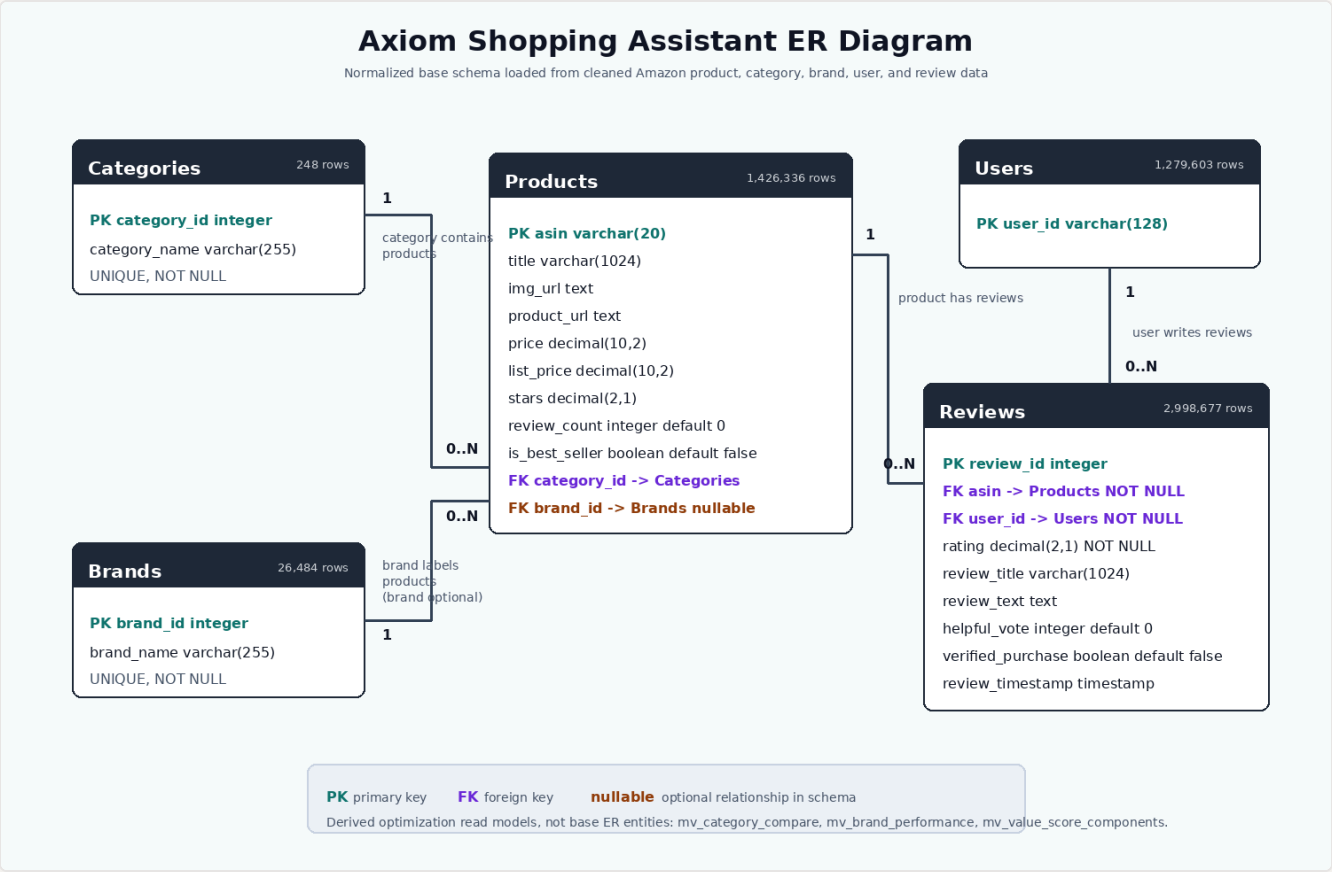


Cleaning, pre-processing, and entity resolution

STAGE	DECISION AND EVIDENCE
Products	Deduplicate by ASIN; parse numeric prices and ratings; normalize FK to `Categories`.
Brand entity resolution	Source has no brand column. Extract canonical brand from title prefix; normalize punctuation and legal suffixes (`Inc.` , `LLC`); drop brands with **< 5 products** to `NULL brand_id` . Result: **26,484 canonical brands**.
Reviews — streamed	Stream CSV chunks; normalize headers across comment + Pure-IDs sources; require `user_id` and a 1–5 rating.
ASIN linking (ER)	Accept `asin` or `parent_asin` ; **fall back to parent ASIN** when child ASIN is not in the catalog; drop orphan reviews.
Review facts	Parse text **and** millisecond-epoch timestamps; clamp helpful votes to ≥ 0; deduplicate by `(asin, user_id, review_timestamp)` .
Output	Normalized CSVs for `Categories` , `Brands` , `Products` , `Users` , `Reviews` , loaded via PostgreSQL `COPY` .

> *Key ER wins: parent-ASIN fallback recovers reviews that would otherwise drop · brand canonicalization collapses thousands of typo variants · review-fact dedup prevents inflated helpful-vote totals.*

Schema design — five normalized base relations



BASE RELATIONS

Categories `category_id`, `category_name`

Brands `brand_id`, `brand_name`

Products `asin` + title, URLs, prices, stars, counts, FKs

Users `user_id`

Reviews `review_id` + ASIN/user FKs, rating, text, helpfulness, timestamp

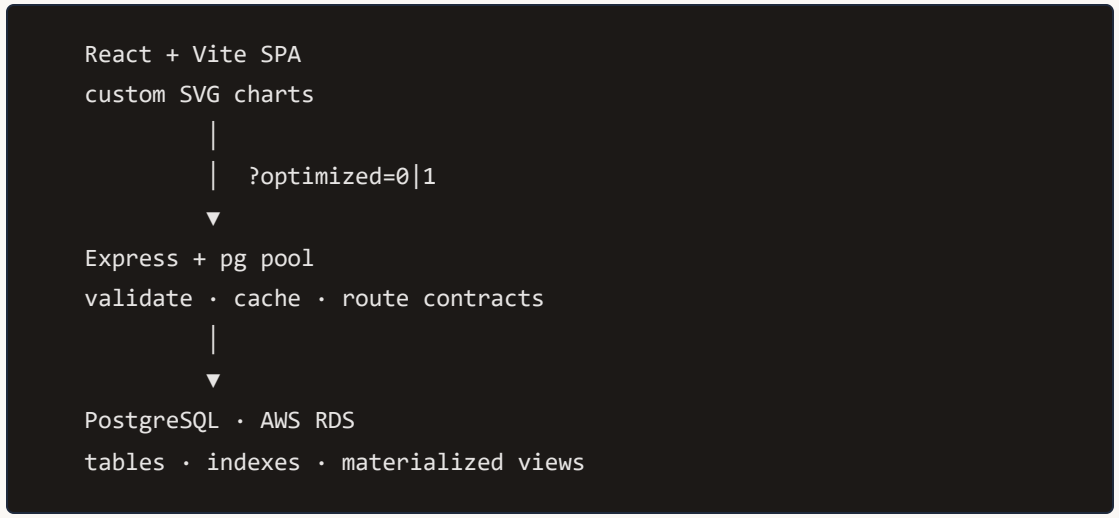
> *Materialized views `mv_value_score_components`, `mv_brand_performance`, `mv_category_compare` are derived read models, not replacement base relations.*

3NF proof — per relation

RELATION	NON-TRIVIAL FDS	3NF JUSTIFICATION
`Categories`	`category_id → category_name`	Determinant is the primary key.
`Brands`	`brand_id → brand_name`	Determinant is the PK after canonicalization; `brand_name` unique.
`Products`	`asin → title, urls, price, list_price, stars, review_count, best_seller, category_id, brand_id`	`asin` is the PK; category and brand **names** live in lookup tables, so no transitive FD through an FK to a non-key display attribute.
`Users`	(no non-key attribute)	Trivially in 3NF.
`Reviews`	`review_id → asin, user_id, rating, title, text, helpful_votes, verified, timestamp`	`review_id` is the PK; product / user details remain in their own tables. The fact table stores only the relationship and review-specific attributes.

> *Every non-trivial FD has a superkey on the LHS — the schema is in 3NF.*

Architecture and query-mode contract



LAYER	TECH
Client	React, Vite, React Router, custom SVG charts
API	Node.js, Express, `pg`
DB	PostgreSQL on AWS RDS
Pipeline	Python, pandas, psycopg2
Test/ops	Vitest, Supertest, benchmark scripts

QUERY-MODE CONTRACT

Every route preserves its public API while supporting:

- `?optimized=0` — original SQL path
- `?optimized=1` — indexed, MV-backed, restructured, or cached path

This is what makes the speedups directly measurable end-to-end — same JSON contract, two SQL plans.

Complex query catalog — eight families

The rubric requires ≥ 4 complex queries drawing on multiple datasets. Axiom ships **eight** families, all touching ≥ 2 base tables.

#	QUERY FAMILY · ROUTE	JOINS	SUBQ / CTE / VIEW	AGGREGATION	UNIVERSAL / EXISTENTIAL	APPLICATION USE
1	Brand performance · `/analytics/brands/performance`	✓	CTE	✓	HAVING	Brand leaderboard
2	Review trend · `/analytics/reviews/trend`	✓	CTE	✓	cond. agg.	Monthly credibility gap
3	Trending products · `/products/trending`	✓	CTE	✓	window filter	Activity-based picks
4	Top value · `/products/top-value`	✓	corr. subq.	✓	**EXISTS**	Above-avg rated, below-avg price
5	Value rankings · `/products/value-rankings`	✓	CTE + MV	✓	weighted score	Custom weighted ranking
6	Helpful reviews · `/products/:asin/helpful-reviews`	✓	lateral / CTE	✓	top-k	Reviewer-context evidence
7	Deals · `/deals`	✓	expr. sort	—	price/rating	Discount discovery
8	Category comparison · `/analytics/categories/compare`	✓	MV-backed	✓	per-category	Analytics chart

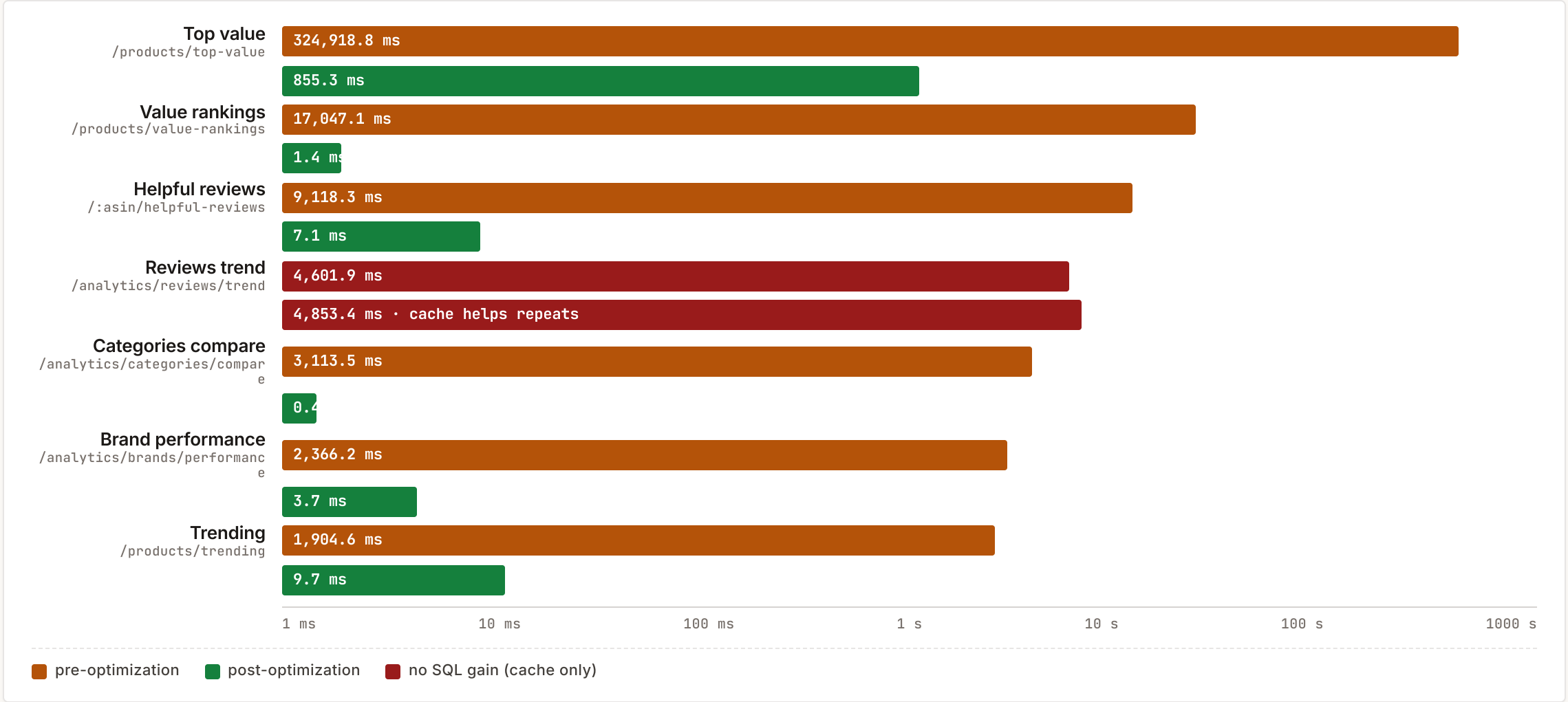
Pre / post optimization timings — all routes

Median ms across 5 measured `EXPLAIN ANALYZE` runs after 1 discarded warmup. Inputs are representative of real UI usage. Source: `docs/timings.md`.

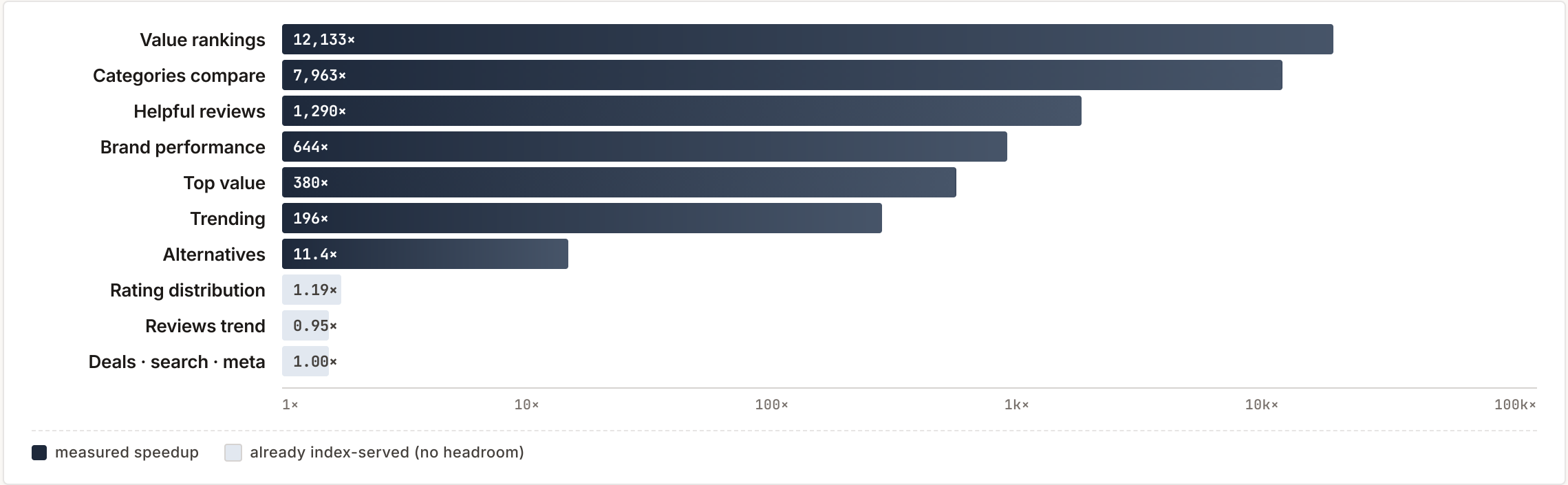
ENDPOINT	INPUT	PRE MS	POST MS	SPEEDUP
`GET /categories`	none	0.4	0.3	1.00×
`GET /brands`	none	6.0	5.9	1.00×
`GET /products/:asin`	`B0178IC734`	0.1	0.1	1.00×
`GET /products/search`	`kasa` , `minStars=4`	1.2	1.2	1.00×
`GET /deals`	`maxPrice=250`	0.5	0.5	1.00×
`GET /products/:asin/rating-distribution`	`B0178IC734`	8.0	6.7	1.19×
`GET /products/:asin/helpful-reviews`	`B0178IC734`	**9,118.3**	**7.1**	**1,290.44×**
`GET /products/:asin/alternatives`	`B0178IC734`	5.6	0.5	11.40×
`POST /cart/savings`	5 ASINs	0.1	0.1	1.00×
`GET /analytics/categories/compare`	none	**3,113.5**	**0.4**	**7,962.92×**
`GET /products/trending`	`Toys & Games`	**1,904.6**	**9.7**	**195.98×**
`GET /products/top-value`	`reviewedSince=2018-01-01`	**324,918.8**	**855.3**	**379.87×**
`GET /analytics/brands/performance`	none	**2,366.2**	**3.7**	**643.68×**
`GET /analytics/reviews/trend`	`Toys & Games`	4,601.9	4,853.4	0.95×
`GET /products/value-rankings`	`0.4/0.2/0.2/0.2`	**17,047.1**	**1.4**	**12,133.17×**

Rubric anchor: two pre-opt runtimes exceed 15 s — top-value (~325 s) and value-rankings (~17 s). All headline optimized runtimes are sub-second.

Optimization results — runtime (log ms)



Optimization results — speedup (log ×)



Four routes exceed 500×; both pre-opt runtimes that crossed the 15 s rubric bar finished sub-second after optimization.

Optimization techniques — rubric four-up

01

Indexing

FK indexes on Products / Reviews; `(asin, review_timestamp)` composite; **partial index** `idx_reviews_verified WHERE verified=TRUE`; **partial expression index** `idx_products_discount` on `(1 - price/list_price)`; trigram title index; **covering index** on the MV default score.

02

Materialized views

`mv_value_score_components` (per-product normalized rating · review-count · price-efficiency · recency, plus a precomputed Balanced score), `mv_brand_performance`, `mv_category_compare`. Refreshed via `scripts/refresh_matviews.js`.

03

Restructuring

Helpful-reviews: rank top-k **before** reviewer-stat aggregation. **Browse lists**: page rows **before** joining display names. **Value rankings**: split Balanced default (indexed) from custom-weight rescore over MV components.

04

Caching

Small in-memory **TTL cache (5 min)** on optimized `/categories`, `/brands`, `/analytics/categories/compare`, and `/analytics/reviews/trend` (keyed by category). Brand performance is MV-served — no route cache needed.

Deep dive · Top value

WHAT IT DOES

For each product, return those **cheaper than their category average** and **rated above their category average**, gated by an **EXISTS** recent-review check parameterized by `reviewedSince`.

WHY IT WAS SLOW

Per-row correlated subqueries computed two category averages and an `EXISTS` over `Reviews` for ≈ 1.4 M products.

OPTIMIZATION

Read precomputed per-category bounds and recent-review markers from `mv_value_score_components`; index the recent-review flag and the `reviewedSince` join column. The **EXISTS** semantics are preserved exactly — moved from query time to MV refresh time.

PRE-OPT → POST-OPT

~~324,918.8 ms~~ → **855.3 ms**

380× MEASURED SPEEDUP

> *From ~5 minutes per request to under 1 second. Pre-opt timing crosses the rubric's 15-second bar by a factor of ~22.*

Optimization stack. Materialized view · covering index · query restructure.

Deep dive · Value rankings 12,133×

WHAT IT DOES

Normalizes four signals — **rating strength**, **review depth**, **price efficiency**, **recent activity** — over the catalog and per-category bounds, then ranks products by a user-weighted score.

WHY IT WAS SLOW

Original query computed global and per-category bounds, normalized four components, joined back to `Products`, and applied weights — **on every request**.

TWO-PATH OPTIMIZATION

1. Balanced preset (`0.4 / 0.2 / 0.2 / 0.2`) uses the indexed precomputed score `idx_mv_value_components_default_score_cover` — pure index range scan.

2. Custom weights rescore **only** the materialized normalized components, skipping the expensive global / per-category aggregation.

PRE-OPT → POST-OPT

~~17,047.1 ms~~ → **1.4 ms**

12,133× MEASURED SPEEDUP

> *Largest measured speedup in the deck. Pre-opt timing crosses the rubric's 15-second bar; post-opt is in the noise floor.*

Optimization stack. Materialized view · covering index · two-path split.

Deep dive · Helpful reviews · Category compare

HELPFUL REVIEWS — RESTRUCTURING

~~9,118.3 ms~~ → **7.1 ms** **1,290×**

Original computed reviewer-aggregate CTE over **all** reviewers for the product.

Optimized path **ranks the top reviews first**, then computes reviewer stats only for that small set (lateral aggregation). Same ranking output, far less work.

Stack. restructure · lateral · top-k first.

CATEGORY COMPARE — MV + CACHE

~~3,113.5 ms~~ → **0.4 ms** **7,963×**

Live aggregate over `Products × Reviews × Categories` replaced by a read from `mv_category_compare`.

A 5-minute route cache further smooths repeat loads of the analytics page.

Stack. materialized view · route cache · refresh-time aggregate.

Honest results — where optimization is modest

ROUTE	RESULT	WHY
`/products/:asin`, `POST /cart/savings`, `/products/search`, `/categories`, `/brands`, `/deals`	≈ 1.00×	Already PK lookups, small lookups, or served by a base-schema index (`idx_products_discount`). No more wins on the hot path.
`/products/:asin/rating-distribution`	1.19×	Already small per-ASIN aggregate. We did not over-engineer it.
`/analytics/reviews/trend`	**0.95×**	Same grouped SQL; planner already chose well for the representative input. The 5-min **route cache** is what helps real users on repeat reads, not the first measured run.

> *We chose not to contrive optimizations on already-cheap queries. Restructuring only happened where `EXPLAIN ANALYZE` showed the planner doing real work — and we report median first-run timings, not cache-warmed numbers.*

Robustness · testing · look & feel

INPUT SANITY

ASIN regex, required keyword/category/ASIN params, numeric and date range checks, zero-sum weight rejection, max 200 ASINs in cart.

SQL SAFETY

Parameterized `pg` bindings throughout — no string concatenation anywhere.

UI ROBUSTNESS

Loading / empty / error states on every API-backed page; **abortable fetches** prevent stale renders after filter changes.

API TESTS

Route, query-mode-selection, cache, validation, and response-shape tests under `server/tests/`.

CLIENT TESTS

API path building, routing, cart behavior, pages, charts, formatting helpers, query-mode toggle.

LOOK & FEEL

Custom SVG bar / horizontal-bar / line charts; layout deliberately distinct from the HW2 starter UI; consistent rating, price, timestamp formatters.

Technical challenges and resolutions

CHALLENGE	RESOLUTION
Product / review entity alignment — child vs. parent ASIN, orphan reviews.	Cleaner accepts both ASIN columns, **falls back to parent ASIN** when the child is missing from the catalog, drops orphans before ingestion.
Brand entity resolution — no brand column; noisy titles.	Conservative title-prefix extraction, punctuation / legal-suffix normalization, canonical display names, **5-product threshold** to avoid one-off entities.
Multi-million-row review files.	Streaming chunked CSV processing, incremental review CSV writes, in-memory user / duplicate tracking, stable FK relationships preserved end-to-end.
Optimization tradeoffs.	Some rewritten SQL was **slower** than the planner's choice. We kept original variants for comparison, used MVs only for stable aggregates, and route-cached only repeat-heavy endpoints.
Source asymmetry.	Documented that the checked-in `amazon_reviews.csv` is a smaller subset than the loaded review corpus; the cleaner discovers additional review streams in `data/raw/`.

Extra credit — deployment

The application is **deployed and reachable by TAs during the live demo.**

FRONTEND

`https://cis550-final-project.onrender.com`

React + Vite static build on Render.

API

`https://team45-cis550-final-project.vercel.app`

Express on Vercel serverless.

DATABASE

AWS RDS PostgreSQL — schema, indexes, materialized views, cleaned CSVs loaded via the data pipeline.

CROSS-SERVICE WIRING

Frontend → API. `VITE_API_BASE_URL = https://team45-cis550-final-project.vercel.app`

API CORS allowlist. `CLIENT_ORIGIN = https://cis550-final-project.onrender.com`

RUBRIC EXTRA CREDIT CLAIMED

Deployment — TAs can access the live site during the presentation. Local fallback is ready as a backup.

Live demo road map

Following `docs/demo_script.md`. Each step calls real routes against the deployed stack with `?optimized=1`.

#	SEGMENT	ROUTES / PAGES	WATCH FOR
1	**Home and search**	`GET /products/search`, query-mode toggle	Keyword search; flip toggle to compare paths.
2	**Product detail**	`/products/:asin` + rating-distribution + helpful-reviews + alternatives	Helpful reviews **9,118** → **7.1 ms** headline.
3	**Cart savings**	`POST /cart/savings`	Aggregate list / current / savings; ASIN-array validation.
4	**Analytics**	category compare, brand performance, review trend	Category compare **3,113** → **0.4 ms**; review trend cache.
5	**Value rankings**	`/products/value-rankings`, Balanced preset + sliders	Value rankings **17,047** → **1.4 ms** (12,133×).
6	**Close**	ER diagram + this deck	Final row counts, strongest speedup, deployment URL.

THANK YOU.

Questions?

Axiom Shopping Assistant · CIS 5500 Final Project · Spring 2026

TEAM 45 · CHENGUANG SHEN · LUIS GARCIA · LEO TAN · ANIKA MADAN